

Visualisatie van de control flow in HLASM-code met Mermaid.js.

Ter ondersteuning van beginnende ontwikkelaars.

Sarah Bader.

Scriptie voorgedragen tot het bekomen van de graad van
Professionele bachelor in de toegepaste informatica

Promotor: Dhr. L. Blondeel

Co-promotor: Dhr. B. Tjassens Keiser

Academiejaar: 2025–2026

Eerste examenperiode

Departement IT en Digitale Innovatie .

**HO
GENT**

Woord vooraf

Tijdens mijn studie kwam ik voor het eerst in contact met mainframe-technologie. Hoewel deze wereld in het begin wat onbekend leek, raakte ik al snel gefascineerd door de enorme capaciteit en de performance van deze systemen. Het feit dat mainframes de ruggengraat vormen van de grootste organisaties ter wereld, gaf mij de motivatie om hier meer over te willen weten.

Tijdens dit proces ontdekte ik HLASM, de assemblytaal van IBM. Mijn interesse werd meteen gewekt omdat deze taal je dwingt om na te denken over de directe werking van de hardware. Mijn overtuiging is dat het leren van een assemblytaal je een betere programmeur maakt. Door op het laagste niveau te werken, krijg je een veel dieper inzicht in hoe instructies en het geheugen echt functioneren. Dit is fundamentele kennis die vaak verloren gaat bij het gebruik van moderne programmeertalen.

HLASM heeft echter een steile leercurve, wat het voor beginners lastig maakt. Omdat ik zelf ook nog volop aan het leren ben, kon ik mij goed verplaatsen in de uitdagingen waar een nieuwe student tegenaan loopt. Vanuit die gedachte ontstond het idee voor deze bachelorproef: het bouwen van een tool die de control-flow van HLASM-programma's visueel voorstelt om het leerproces te ondersteunen.

Het uitwerken van dit project heeft mij niet alleen technische kennis opgeleverd, maar ook inzicht gegeven in hoe complexe informatie vereenvoudigd kan worden. Het bouwen van de proof-of-concept was een waardevolle kans om theorie om te zetten in iets concreets.

Dankwoord

Graag wil ik mijn promotor, L. Blondeel, bedanken voor de begeleiding en de nuttige feedback tijdens het schrijven van deze bachelorproef.

Daarnaast bedank ik mijn co-promotor, B. Tjassens Keiser, voor de hulp bij technische vragen.

Tot slot wil ik mijn familie en vrienden bedanken voor hun steun en motivatie. Hun aanmoedigingen hebben mij geholpen om dit traject succesvol af te ronden.

Samenvatting

Mainframes spelen nog steeds een belangrijke rol in sectoren zoals banken, verzekeringen en luchtvaart. Veel van de software op deze systemen is geschreven in High Level Assembler (HLASM). Deze programma's zijn vaak moeilijk te begrijpen, zeker voor beginners. Daarnaast is er een tekort aan nieuwe ontwikkelaars, wat het doorgeven van kennis moeilijk maakt.

Deze bachelorproef onderzoekt hoe visualisatie kan helpen bij het begrijpen van de control-flow van HLASM-programma's. De centrale onderzoeksvraag luidt: "Hoe effectief versnellen statische analyse en Mermaid-diagrammen het leerproces van HLASM bij beginners?". Het doel van het onderzoek is het ontwikkelen en evalueren van een proof-of-concept (PoC) die HLASM-code omzet naar een visuele voorstelling.

Het onderzoek werd uitgevoerd in verschillende stappen. Eerst werd een literatuurstudie gedaan naar mainframes, HLASM en bestaande visualisatietechnieken. Daarna werd HLASM-code geanalyseerd om te bepalen welke onderdelen belangrijk zijn voor de control-flow, zoals labels en sprongen. Op basis daarvan werd een proof-of-concept ontwikkeld als webapplicatie. Deze tool zet HLASM-code via statische analyse om naar een flowchart in Mermaid.js. Tot slot werd de tool getest met vier beginnende gebruikers.

Uit de resultaten blijkt dat de visualisatie vooral helpt om sneller een overzicht te krijgen van de structuur van een programma. De deelnemers konden gemakkelijker zien hoe verschillende delen van de code met elkaar verbonden zijn en waar vertakkingen plaatsvinden.

Tegelijk werd duidelijk dat de tool het begrijpen van de code niet volledig vervangt. Om de werking van het programma correct te begrijpen, blijft het nodig om de code zelf te lezen en te analyseren. De visualisatie werd dus vooral gezien als een hulpmiddel dat het proces ondersteunt.

Daarnaast bleek dat de meerwaarde van de tool groter is bij grotere programma's. Bij kleinere codefragmenten is de structuur meestal eenvoudig genoeg om manueel te volgen, waardoor de visualisatie minder toevoegt.

De resultaten tonen aan dat statische analyse in combinatie met een visuele voorstelling een nuttig hulpmiddel kan zijn voor beginnende ontwikkelaars. De ontwikkelde PoC heeft echter beperkingen: slechts een deel van de HLASM-instructies wordt ondersteund en complexe situaties zoals macro's en indirecte sprongen worden niet meegenomen.

Samenvattend kan gesteld worden dat een eenvoudige visualisatietool helpt om de structuur van HLLASM-programma's sneller te begrijpen. De tool ondersteunt het leerproces, maar vervangt het niet, en is vooral nuttig bij grotere programma's.

Inhoudsopgave

Lijst van figuren	ix
Lijst van tabellen	x
Lijst van codefragmenten	xi
1 Inleiding	1
1.1 Probleemstelling	1
1.2 De onderzoeksvraag	2
1.3 Onderzoeksdoelstelling	2
1.4 Het verwachte resultaat	3
1.5 Structuur van het onderzoek	3
2 Literatuurstudie	4
2.1 De mainframeindustrie	4
2.2 Aanwezigheid van HLASM-code	5
2.2.1 Onderhoud en Modernisatie	5
2.2.2 Bestaande HLASM-hulpmiddelen	5
2.2.3 Kosten van Onderhoud	7
2.3 Moeilijkheden voor Beginnende Ontwikkelaars	7
2.3.1 Eigen Bijdrage	8
2.4 Grafische Softwarevisualisatie	8
2.4.1 Control-Flow Grafieken	8
2.4.2 Mermaid.js	8
2.4.3 Eigen Bijdrage	8
2.5 Parsing van programmeertalen	9
2.5.1 JavaScript Parsing	9
3 Methodologie	10
3.1 Fase 1: Literatuurstudie	10
3.2 Fase 2: Technische analyse van HLASM	10
3.3 Fase 3: Requirementsanalyse	11
3.4 Fase 4: Ontwerp en Implementatie	11
3.5 Fase 5: Evaluatie	12
4 HLASM-analyse	13
4.1 Inleiding	13

4.2	De kolomgebaseerde structuur	15
4.2.1	Voorbeeld	15
4.3	De control-flow	16
4.3.1	Branching	17
4.3.2	Relative Branching J-instructies	18
4.3.3	Compare-and-Branch (CGIJ)-instructies	18
4.3.4	Instructies voor Control-Flow Mapping	19
5	Control-flow visualisatie	21
5.1	Requirements	21
5.1.1	Functionele vereisten	21
5.1.2	Niet-functionele vereisten	22
5.2	Ontwerp	22
5.2.1	Mapping van HLASM naar Mermaid.js	23
6	Proof-of-Concept	24
6.1	Architectuur van de applicatie	24
6.1.1	Stap 1: Parsing	25
6.1.2	Stap 2: Control-flow analyse	26
6.1.3	Stap 3: Visualisatie	27
6.1.4	Gebruikersinterface	28
6.1.5	Gebruik van de applicatie	29
6.2	Beperkingen van de implementatie	29
6.3	Evaluatie	30
6.3.1	Resultaten	30
7	Conclusie	31
A	Onderzoeksvoorstel	33
A.1	Inleiding	33
A.2	Literatuurstudie	34
A.2.1	Belang van de mainframecomputer	34
A.2.2	Nood aan onderhoud van HLASM-code	35
A.2.3	Moeilijkheden voor beginnende ontwikkelaars	35
A.2.4	Effectiviteit van visualisatietechnieken	35
A.2.5	Bestaande visualisatietools en parsers	35
A.2.6	Control Flow in High Level Assembler	36
A.2.7	JavaScript Parsing	36
A.2.8	Mermaid.js en flowcharts	37
A.3	Methodologie	37
A.3.1	Fase 1: Oriëntatie en Literatuurstudie	37
A.3.2	Fase 2: Requirementsanalyse	38
A.3.3	Fase 3: Implementatie en Evaluatie	38

A.3.4	Fase 4: Resultaat en Verdediging	39
A.3.5	Tijdsplanning	39
A.4	Verwacht resultaat, conclusie	39
Bibliografie		40

Lijst van figuren

2.1	De FermaT Maintenance Environment (FME).	6
4.1	De opstelling van een HLASM-programma.	14
6.1	Invoer van HLASM-code in de webapplicatie.	28

Lijst van tabellen

4.1	De kolomgebaseerde structuur van een hlasm-instructie.	15
4.2	De eigenschappen van de mnemonic 'AR' (2025).	16
4.3	De mogelijkheden van de CC na de 'AR' instructie (<i>z/Architecture Principles of Operation</i> , 2022).	16
4.4	De eigenschappen van de mnemonic 'CR' (2025).	16
4.5	De mogelijkheden van de CC na de 'CR' instructie (Ehrman, 2001). . . .	17
4.6	Relevante HLASM-instructies voor control-flow mapping.	20
5.1	Mapping van HLASM-elementen naar Mermaid.js	23

Lijst van codefragmenten

4.1	Standaard Linkage Setup (Bosworth, 2009).	15
4.2	Voorbeeld van vertakkingen in HLASM.	18
4.3	Voorbeeld van relative branching met J-instructies.	18
4.4	Voorbeeld van een CGIJ-instructie.	19
6.1	De HLASM parser.	25
6.2	De code die instructies in blokken verdeelt.	26
6.3	De conversie naar mermaid.js.	27

1

Inleiding

1.1. Probleemstelling

Vaak wordt een mainframe gezien als verouderde technologie die enkel gekend is uit films. Het tegengestelde is waar: mainframes zijn één van de meest geavanceerde servers die voortdurend onderhouden en gemoderniseerd worden. Mainframes zijn dan ook nog volop in gebruik. Door de betrouwbaarheid en capaciteit van een mainframe is het vooral geschikt voor grote aantallen transacties/aanvragen van gebruikers. Mainframes worden vaak gebruikt door banken, verzekeraars, luchtvaartmaatschappijen... om hun fundamentele bedrijfsprocessen te ondersteunen.

Omdat deze systemen zo fundamenteel zijn, is het onderhoud ervan van groot belang. Vaak zijn de programma's die draaien op het mainframe ook tientallen jaren oud. Een deel van deze programma's is geprogrammeerd in High Level Assembler (HLASM). Deze programmeertaal wordt vaak gekozen omdat het programma snel moet zijn en directe controle over de hardware nodig heeft. Andere programma's maken vaak gebruik van de HLASM-programma's, waardoor ze ervan afhankelijk worden.

Daarbij zijn deze programma's vaak minimaal gedocumenteerd of bevatten ze verouderde documentatie. Hierdoor is het moeilijk om als nieuwe mainframe developer de voorgaande code te begrijpen.

Veel bedrijven hadden het idee dat mainframes onnodig zouden worden en dat ze konden vertrouwen op "modernere" servers. In werkelijkheid groeide hun workload alleen maar en werd die te groot om op reguliere servers te runnen. Met de voortdurende modernisatie bleek het mainframe echter de beste keuze om hun bedrijfsactiviteiten verder op te vertrouwen.

Met de oude mindset dat ze van het mainframe zouden weggaan, investeerden bedrijven niet in nieuw mainframe talent. Hierdoor bestaat de industrie vooral uit professionals die binnenkort met pensioen gaan. Nu het duidelijk is dat een mainframe beter in staat is om hun huidige workload te beheren, is het tekort aan junior talent een probleem. De kennis van de bedrijfsprogramma's en systemen zit dus in een generatie die de arbeidsmarkt verlaat.

Hierdoor is er nood aan een nieuwe generatie om vooral het onderhoud van de systemen en programma's te beheren. Door het gebrek aan voldoende en bruikbare documentatie is het vaak moeilijk om als nieuwe mainframe programmeur code te begrijpen, wat het onderhouden ervan belemmert. Documentatie is dus belangrijk voor de leesbaarheid en onderhoudbaarheid van de code, waardoor minder fouten worden gemaakt.

Een hulpmiddel zou een visualisatietool kunnen zijn die kan dienen als documentatie. Beginners hebben vaak moeite met het begrijpen van de control flow van een programma, daarom is visualisatie van de control flow vooral handig.

1.2. De onderzoeksvraag

Dit onderzoek is gericht op het beantwoorden van de volgende vraag:

Hoe effectief versnellen statische analyse en Mermaid-diagrammen het leerproces van HLASM bij beginners?

Binnen dit onderzoek wordt gefocust op twee aspecten: het technische en het menselijke. Er wordt gekeken naar hoe de visualisatie technisch kan geïmplementeerd worden en ook naar de effectiviteit van de visualisatie als documentatie en hulpmiddel bij het begrijpen van de control flow in HLASM.

1.3. Onderzoeksdoelstelling

Het doel van dit onderzoek is het uitwerken van een Proof-of-Concept en een bijhorende analyse ervan. De PoC moet in staat zijn de belangrijkste onderdelen van HLASM-code om te zetten in een Mermaid.js flowchart. De focus ligt expliciet op de structuur van het programma, dus minder op de details. Het moet bijvoorbeeld weergeven waar het programma begint, waar het zich opsplitst in verschillende takken en waar het eindigt.

Vooraleer de implementatie van start gaat, wordt onderzoek gedaan naar de bestaande technologieën die van hulp kunnen zijn. Dit onderzoek wordt uitgevoerd in de vorm van een literatuurstudie. Tijdens de literatuurstudie wordt gekeken naar het probleemdomein en het oplossingsdomein.

Binnen de literatuurstudie worden volgende vragen gesteld om het probleemdo-
mein beter te verstaan:

- Waarom is een mainframe nog steeds van belang?
- Waarom is er nood aan het onderhouden van HLASM-code?
- Hoe helpt visualisatie met het begrijpen van codestructuur?
- Wat zijn de moeilijkheden bij het begrijpen van assemblytalen voor beginnende ontwikkelaars?

Rond het oplossingsdomein worden volgende vragen bestudeerd:

- Hoe helpt visualisatie bij het sneller begrijpen van programma's?
- Welke tools bestaan er al om HLASM-code te visualiseren?
- Welke HLASM parsers bestaan er?

1.4. Het verwachte resultaat

Het verwachte resultaat van dit onderzoek is dat de visualisatie helpt bij het beheer-
sen van HLASM-control flow, in tegenstelling tot het gebruik van manuele metho-
des. Om dit resultaat te meten wordt de PoC geëvalueerd aan de hand van testen
die de tijd meten met de PoC en met de manuele manier.

1.5. Structuur van het onderzoek

Deze bachelorproef is als volgt opgesteld:

In Hoofdstuk 2 (Literatuurstudie) worden de deelvragen uit het oplossingsdomein
en het probleemdo-
mein onderzocht aan de hand van bestaande onderzoeken en
literatuur. Hiermee wordt een beter zicht gecreëerd op het probleem en de oplos-
sing om zo gericht mogelijk te werken.

In Hoofdstuk 3 (Methodologie) wordt toegelicht welke onderzoekstechnieken wor-
den gebruikt.

In Hoofdstuk 4 (HLASM-analyse) wordt gekeken naar Assembler, hoe de program-
meertaal is opgebouwd en welke elementen de structuur van het programma vast-
leggen.

In Hoofdstuk 5 (Control-flow visualisatie) wordt bekeken hoe de Assembler-structuur
kan worden omgezet naar Mermaid.js-elementen. Er wordt onderzocht hoe dit vi-
sueel logisch kan worden gerepresenteerd.

In Hoofdstuk 6 (Proof-of-Concept) wordt de PoC uitgewerkt en daarna geëvalueerd
aan de hand van testen.

In Hoofdstuk 7 (Conclusie) wordt gekeken naar het resultaat van de evaluatie en
naar de literatuurstudie om een antwoord te formuleren op de onderzoeksvragen.

2

Literatuurstudie

2.1. De mainframeindustrie

Voor een lange tijd werd beweerd dat de mainframecomputer zou uitsterven en dat grote bedrijven er afstand van zouden doen (Sagers e.a., [2018](#)). Het einde van mainframes is nog steeds niet gekomen en in sommige sectoren is er zelfs sprake van een trend om te blijven investeren in mainframes (Sagers e.a., [2018](#)).

Het onderzoek van Sagers e.a. ([2018](#)) toont aan dat er een behoefte is aan een voortdurende werkracht met mainframe-kennis. Mainframes bekleden nog steeds een belangrijke rol in bedrijven en dat zal waarschijnlijk nog tientallen jaren zo blijven. Ook wordt vermeld dat de academische wereld niet in staat is om voldoende mensen met mainframe-kennis op te leiden. Dit komt omdat de opleidingen in het verleden zijn stopgezet. Uit het onderzoek blijkt dat het aantal nieuwe medewerkers dat de industrie nodig heeft als gevolg van pensionering, groter is dan het aantal mensen dat wordt opgeleid door de academische wereld (Sagers e.a., [2018](#)). Het onderzoek van Sagers e.a. ([2018](#)) wordt ondersteund door recent onderzoek waarin wordt geconstateerd dat een mainframe voor grote IT-bedrijven nog steeds een belangrijke functie vervult. De mainframecomputer heeft daarbij ook nieuwe mogelijkheden gekregen die het aantrekkelijk maken voor nieuwe kopers. Het vermogen van mainframes om de fundamentele bedrijfsprocessen voor banken en verzekeringsmaatschappijen te waarborgen, samen met de nieuwe mogelijkheden die het de afgelopen jaren heeft gekregen, zorgen ervoor dat mainframes relevant blijven (Shaik, [2024](#)).

Daarnaast wordt in het onderzoek van Waites en Ketterer ([2013](#)) het punt herhaald dat de werknemers die mainframes ontwikkelen, ondersteunen en onderhouden bijna met pensioen gaan en daarmee hun waardevolle kennis verdwijnt.

2.2. Aanwezigheid van HLASM-code

Veel software die vandaag op mainframes draait wordt beschouwd als legacysoftware. Programma's die vaak tientallen jaren geleden ontwikkeld zijn maar nog steeds een cruciale rol spelen binnen organisaties. Legacysoftware bevat meestal een grote hoeveelheid bedrijfslogica die moeilijk te reconstrueren is wanneer het systeem volledig zou worden vervangen (Sneed & Verhoef, 2019).

Door de hoge nauwkeurigheid, flexibiliteit en efficiëntie van HLASM, blijft het een essentieel component in mainframe-omgevingen (Yalamanchili, 2022). Het zorgt ervoor dat systeemontwikkelaars toegang hebben tot geheugen en het kunnen manipuleren. In hogere programmeertalen zoals COBOL, zijn deze functionaliteiten beperkt. HLASM wordt dus gebruikt binnen een context waar prestatie centraal staat, hierdoor blijft het een belangrijk component (Yalamanchili, 2022).

2.2.1. Onderhoud en Modernisatie

Applicaties moderniseren en onderhouden is een mogelijkheid die grote voordelen en lagere kosten kan bieden (Ehrman, 2004). Het legacy-systeem kan gezien worden als een investering die je hergebruikt. Ook heeft het minder nood aan nieuwe testen, sinds de code minder wordt veranderd integeningstelling tot een volledige conversie (Ehrman, 2004).

Bij programmeertalen zoals assembler kan program comprehension complex zijn. De code bestaat uit elementaire instructies die direct met de hardware werken. Hierdoor is het moeilijker om de logica van het programma te identificeren zonder extra hulpmiddelen zoals analysetools of visualisaties (Baldwin e.a., 2009).

Tools kunnen helpen met het moderniseren van HLASM-applicaties (Ehrman, 2004). Ze kunnen namelijk meer inzicht geven in het gedrag van applicaties. Het moderniseren van HLASM kan stapsgewijs aangepakt worden: afzonderlijke instructies, blokken code, procedures, modules... De organisatie kiest zelf hoe groot of klein deze stappen zijn en per stap kunnen de nodige testen uitgevoerd worden (Ehrman, 2004).

2.2.2. Bestaande HLASM-hulpmiddelen

High Level Assembler Toolkit Feature

De High Level Assembler Toolkit Feature is een IBM-product dat een set van tools bevat om het herstel en de ontwikkeling van HLASM applicaties te verbeteren, analyseren, debuggen en te onderhouden (Ehrman, 2001). Hierdoor helpt het bij de modernisatie van HLASM-applicaties (Ehrman, 2004).

De tool is voornamelijk gericht op professionele mainframeontwikkelaars en vereist het gebruik van een volledige mainframeontwikkelomgeving. De proof-of-concept richt zich specifiek op het verbeteren van programma comprehension door middel van visuele representaties van HLASM-code.

Tape-z

Tape/Z is een toolkit die werd ontworpen voor het analyseren en visualiseren van HLASM-code. Het doel van deze tool is om code overzichtelijk in kaart te brengen door middel van het parsen van instructies en het automatisch opbouwen van control-flow diagrammen die de structuur van het programma weergeven (avishek-sen-gupta, 2025).

Tape/Z biedt veel nuttige functionaliteiten voor het analyseren van HLASM-code, maar het gebruik van de tool vereist een technische configuratie. Dit kan een drempel vormen voor ontwikkelaars of studenten die snel een visuele representatie willen genereren.

De proof-of-concept heeft als doel om een meer toegankelijke tool te zijn. De PoC wordt een webapplicatie die HLASM-code rechtstreeks omzet naar Mermaid.js syntax. Dit maakt het mogelijk om diagrammen te bouwen met tekstuele syntax die eenvoudig aangepast en uitgebreid kan worden. Hierdoor kunnen gebruikers de gegenereerde visualisaties niet alleen bekijken, maar ook verder aanpassen.

Eigen Bijdrage

De focus van deze studie ligt op het ontwikkelen van een lichtgewicht en toegankelijke tool die specifiek gericht is op inzicht in control-flow. In tegenstelling tot bestaande oplossingen wordt gekozen voor een webgebaseerde aanpak.

2.2.3. Kosten van Onderhoud

In het onderzoek van Xia e.a. (2018) werd een grootschalig veldonderzoek gedaan naar hoe programmacomprehensie in de praktijk wordt uitgevoerd. Er werd gekeken naar 79 ontwikkelaars die aan 7 industriële projecten werkten. Als resultaat werd geconstateerd dat ontwikkelaars gemiddeld tot 58% van hun tijd besteden aan het begrijpen van programma's. Hierbij gebruiken ze vaak webbrowsers en documentatie editors. Ook werd vastgelegd dat senior ontwikkelaars minder tijd besteden aan het begrijpen van programma's en dat er meer tijd nodig voor is tijdens een onderhoudsfase (Xia e.a., 2018).

2.3. Moeilijkheden voor Beginnende Ontwikkelaars

Assemblytalen zijn zeer gedetailleerd, als ontwikkelaar definieer je elke kleine stap van het programma, wat ingewikkeld kan zijn voor beginners. Uit onderzoek blijkt dat naarmate de ervaring toeneemt, het gemakkelijker wordt om de functionaliteit van code te begrijpen (Feigenspan e.a., 2012).

Binnen het onderwijs hebben studenten vooral moeite met de componenten die de logische volgorde van het programma bepalen. Dit zijn de componenten die de control flow van het programma definiëren. Bij het begrijpen van de control flow maken beginners vaak fouten en ervaren ze vaak verwarring (Kawash, 2024).

2.3.1. Eigen Bijdrage

Visualisatie van control-flow kan deze moeilijkheden verminderen door de structuur van programma's zichtbaar te maken. Deze studie is specifiek om een tool te ontwikkelen die control-flow grafisch weergeeft vanuit HLASM-code.

2.4. Grafische Softwarevisualisatie

In het onderzoek van Cárdenas en Aponte (2017) werd een gecontroleerd experiment uitgevoerd om de impact van het gebruik van grafische visualisatietechnieken bij het begrijpen van software vast te leggen. Hiervoor werden tamen uitgevoerd door 20 studenten, de helft gebruikte een visualisatietool, de andere helft niet. Er werd beoordeeld op de voltooiingstijd en de nauwkeurigheid van de antwoorden. Uit het onderzoek werd geconcludeerd dat het gebruik van grafische visualisatie de juistheid verhoogt met 21,45%. Er was geen bewijs dat er een vermindering was van voltooiingstijd. Grafische visualisatietechnieken kunnen dus bijdragen aan het begrijpen van programma's (Cárdenas & Aponte, 2017).

2.4.1. Control-Flow Grafieken

Een control-flow grafiek heeft knooppunten die overeenkomen met basisblokken code met elk een gerichte rand die een mogelijke controlestroom tussen twee knooppunten aangeeft (Debbabi e.a., g.d.). Hierdoor is het mogelijk om alle uitvoeringspaden van een programma te analyseren. Ze zijn vooral heel nuttig voor statische analyse en om inzicht te verkrijgen in de structuur van een programma (Debbabi e.a., g.d.).

2.4.2. Mermaid.js

Mermaid.js is een open-source tool waarmee diagrammen en flowcharts worden opgesteld vanuit tekst. Het heeft een simpele syntax die het makkelijk maakt om visualisaties te maken. De tool wordt gebruikt voor documentatie, code-review en visuele uitleg (Sveidqvist & Jain, 2021).

2.4.3. Eigen Bijdrage

Binnen deze studie wordt Mermaid.js gebruikt als outputformaat. Hierdoor kunnen gegenereerde diagrammen eenvoudig aangepast worden en kan het geïntegreerd worden in documentatie.

2.5. Parsing van programmeertalen

Parsing is een fundamenteel onderdeel van de analyse en verwerking van programmeertalen. Tijdens het parsingproces wordt broncode geanalyseerd volgens een grammatica om de syntactische structuur van het programma te bepalen. Dit proces is een belangrijke stap binnen compilers, interpreters en analysetools (Zaytsev, 2020).

Een programmeertaal wordt meestal beschreven door grammaticaregels. Op basis van deze regels kan een parser bepalen of een stuk broncode syntactisch correct is en hoe de verschillende onderdelen van het programma met elkaar verbonden zijn. Het resultaat is vaak een boomstructuur, zoals een abstract syntax tree (AST), waarin de hiërarchische structuur van het programma wordt weergegeven (Sun e.a., 2023).

2.5.1. JavaScript Parsing

JavaScript is een geschikte programmeertaal om een parser mee te bouwen omdat het veel libraries heeft en makkelijk integreert met webtechnologieën (Contributors, 2023). Daardoor kan de parsinglogica gebruikt worden voor analyse en directe visualisatie in een webinterface. JavaScript heeft ook een brede community en documentatie die het makkelijker maken om een parser te ontwikkelen (Contributors, 2023).

3

Methodologie

Binnen dit hoofdstuk wordt toegelicht hoe dit toegepast en technisch onderzoek werd uitgevoerd. Het onderzoeksproces werd uitgevoerd in verschillende fasen. Per fase wordt de doelstelling, de gebruikte onderzoeksmethode en het bijhorende resultaat toegelicht.

3.1. Fase 1: Literatuurstudie

Binnen de literatuurstudie werd onderzoek gedaan naar het probleemdomain en het oplossingsdomain. De doelstelling was om inzicht te krijgen in het belang van mainframes, de rol van HLASM en de moeilijkheden bij het begrijpen van assembly-talen voor beginnende ontwikkelaars. Daarnaast werd onderzocht welke visualisatietechnieken en bestaande tools gebruikt worden voor het analyseren van control flow.

De onderzoeksmethode in deze fase bestond uit het analyseren van wetenschappelijke literatuur, technische documentatie en bestaande tools.

Het resultaat van deze fase is Hoofdstuk 2 (Literatuurstudie), waarin de deelvragen worden beantwoord en de noodzaak van een visualisatietool wordt onderbouwd.

3.2. Fase 2: Technische analyse van HLASM

In deze fase werd de structuur van HLASM-broncode geanalyseerd. De doelstelling was het identificeren van structurele elementen die de control flow representeren, zoals labels, conditionele vertakkingen en sprongen.

De methode bestond uit een technische analyse van representatieve HLASM-codefragmenten. Hierbij werd specifiek gekeken naar de kolomgebaseerde structuur en naar patronen die gebruikt kunnen worden voor statische analyse.

Het resultaat van deze fase is Hoofdstuk 4 (HLASM-analyse), waarin wordt vastgelegd welke structurele elementen noodzakelijk zijn voor de visualisatie van de control flow.

3.3. Fase 3: Requirementsanalyse

Op basis van de literatuurstudie en de technische analyse werd een requirementsanalyse opgesteld. De doelstelling was het definiëren van een afgebakende en technisch haalbare scope voor de proof-of-concept.

Volgende aspecten werden bepaald:

- Welke subset van HLASM-instructies representeert de control flow?
- Welke Mermaid.js-syntax vertaalt zich naar de juiste visualisatie van de control flow?
- Wat zijn de meetbare succescriteria voor de evaluatiefase?

Het resultaat van deze fase is Hoofdstuk 5 (Control-flow visualisatie), waarin een duidelijke afbakening en functionele eisen voor de verdere ontwikkeling en evaluatie staan beschreven.

3.4. Fase 4: Ontwerp en Implementatie

In deze fase werd de proof-of-concept ontwikkeld. De doelstelling was het automatisch omzetten van een geselecteerde subset HLASM-code naar een control-flowdiagram in Mermaid.js-syntax

De ontwikkeling gebeurde iteratief. Er werd een JavaScript-gebaseerde parser ontworpen die via patroonherkenning instructies en labels detecteert in de HLASM-broncode. Vervolgens werden deze gemapt naar Mermaid.js-elementen.

Het resultaat van deze fase is Hoofdstuk 6 (Proof-of-Concept), waarin het ontwerp en de implementatie wordt weergegeven.

3.5. Fase 5: Evaluatie

In de evaluatiefase werd onderzocht of de proof-of-concept effectief bijdraagt aan het sneller en correcter begrijpen van HLASM-control flow.

Er werd een vergelijkende studie uitgevoerd met 4 beginnende mainframeontwikkelaars. De deelnemers analyseerden HLASM-codefragmenten, met of zonder gebruik van de PoC.

Om de correctheid te kunnen meten, werden volgende criteria vastgelegd:

- het identificeren van start- en eindpunten,
- het aanduiden van alle mogelijke vertakkingen,
- het uitleggen van een bepaalde conditionele vertakking.

Naast correctheid werden de resultaten ook geëvalueerd op de benodigde tijd om de code te analyseren.

Het resultaat van deze fase wordt besproken in Hoofdstuk 6 (Proof-of-Concept) en geïnterpreteerd in Hoofdstuk 7 (Conclusie).

4

HLASM-analyse

In dit onderzoek worden de termen assembleren "HLASM" gebruikt als synoniemen voor de High Level Assembler van IBM voor z/Architecture. De analyse wordt uitgevoerd op basis van de verkregen kennis vanuit Ehrman (2001) en *High Level Assembler for z/OS & z/VM & z/VSE Language Reference Version 1 Release 6* (g.d.).

4.1. Inleiding

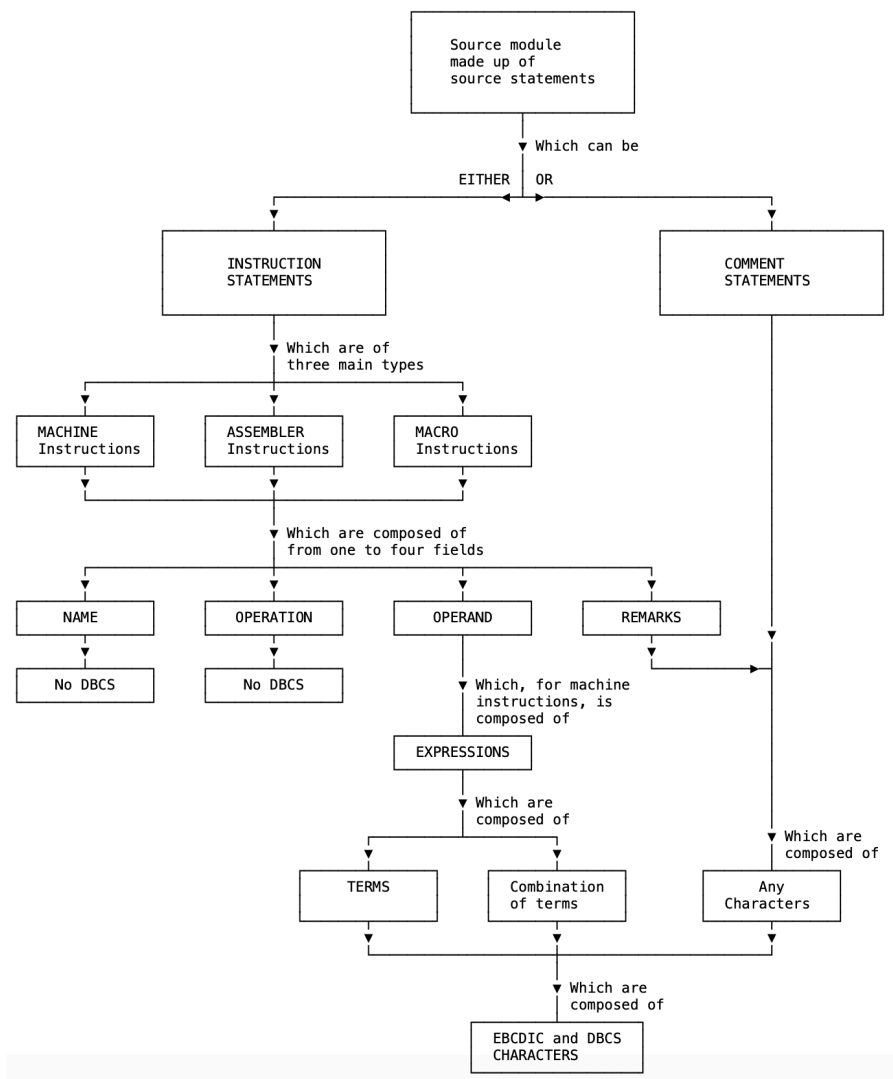
High Level Assembler (HLASM) is een low-level programmeertaal voor IBM mainframes. Een HLASM-programma bestaat uit een reeks instructies die direct met de hardware interageren. Er zijn vier soorten instructies: commentaarinstructies, machine-instructies, assemblerinstructies en macroinstructies.

- **Commentaarinstructies;** Deze instructies bieden toelichting in het programma voor de leesbaarheid en begrijpbaarheid.
- **Machine-instructies;** Deze instructies worden omgezet naar in machinetaal-instructies, die de cpu uitvoert wanneer het programma wordt uitgevoerd.
- **Assemblerinstructies;** Deze instructies geven informatie aan de assembler. Ze kunnen bijvoorbeeld gebruikt worden om gegevens te genereren of een titel te geven aan de uitvoer.
- **Macroinstructies;** Deze instructies worden gebruikt als referentie naar een groep van instructies. De assembler vervangt die referentie automatisch naar de groep instructies.

De bovenvermelde instructies, met uitzondering tot commentaarinstructies, bestaan elk uit vier delen die velden/fields worden genoemd.

- The name/label field
- The operation/mnemonic field (altijd verplicht)
- The operand field (bij meerdere operands zijn deze gescheiden door een komma)
- The remarks field

Elk van deze velden zijn gescheiden met één of meerdere spaties.



Figuur 4.1: De opstelling van een HLASM-programma (*High Level Assembler for z/OS & z/VM & z/VSE Language Reference Version 1 Release 6, g.d.*).

4.2. De kolomgebaseerde structuur

De Assembler verwerkt records van precies 80 bytes lang, dus dat betekent 80 tekens in één lijn code. Het aantal tekens wordt gerepresenteerd als kolommen. De instructies nemen enkel de posities 1 tot en met 71 in beslag. Daarnaast, dient kolom 72 om mee te geven als de volgende lijn code een voortzetting is van de huidige of een volledig nieuwe regel. Wanneer een niet-leeg teken ingevuld is, betekent dit dat het een voortzetting is. Bij een voortzetting, start de volgende regel vanaf kolom 16. De kolommen 73 tot 80 worden door de assembler genegeerd.

De kolommen die in beslag worden genomen door de instructies (1 - 71), zijn nogmaals opgedeeld voor de verschillende velden:

Kolommen	Inhoud
1 - 8	De name/label
9	Blank
10 - 14	De operartion/mnemonic
15	Blank
16 - 71	De operands(s), gescheiden door komma's - hier mogen geen spaties voorkomen (enkel tussen quotes). Een spatie beëindigt het operand-veld, alles daarna behoort tot het remarks-veld.

Tabel 4.1: De kolomgebaseerde structuur van een hlasm-instructie.

4.2.1. Voorbeeld

In het voorbeeld is de kolomstructuur af te leiden, de liniaal dient enkel als visueel hulpmiddel. In kolom 1 staat het name-veld of een asterisk voor commentaarregels. Vanaf kolom 10 staat het operation-veld, gevolgd door het operand-veld in kolom 16 als de parameters. De tekst die na de operanden volgt, met minimaal één spatie tussen, is het remarks-veld.

```

1 *****
2 * PROGRAMMER:  EDWARD BOSWORTH, COLUMBUS STATE UNIVERSITY *
3 *****
4 ----+----1----+----2----+----3----+----4----+----5----+----6----+----7--
5 LAB1      CSECT
6           SAVE  (14,12)                SAVE THE CALLER'S REGISTERS
7           BALR  R12,0                    ESTABLISH
8           USING *,R12                    ADDRESSABILITY
9           LA    R2,SAVEAREA              ADDRESS OF MY SAVE AREA
10          ST    R2,8(,R13)                FORWARD CHAIN MINE
11          ST    R13,SAVEAREA+4            BACKWARD CHAIN CALLER'S
12          LR     R13,R2                    SET 13 FROM MY SUB CALLS

```

Listing 4.1: Standaard Linkage Setup (Bosworth, 2009).

4.3. De control-flow

Aan de hand van eerdere berekeningen of een logische vergelijking kunnen sprong-instructies alternatieve routes creëren. De uitkomst van een toetsing wordt aangegeven in de conditiecode. De Condition Code (CC) is een veld van twee bits binnen het Program Status Word¹ (PSW) die vier mogelijke waarden kan bevatten. Afhankelijk van de operatie hebben deze waarden een bepaalde betekenis.

In het volgende voorbeeld wordt gekeken naar hoe de 'AR' instructie de condition code aanpast.

Mnemonic	Operands	Opcode	Instruction
AR	R,R	1A	ADD (32)

Tabel 4.2: De eigenschappen van de mnemonic 'AR' (2025).

Deze instructie telt de waarden in twee registers² bij elkaar op, bewaart de som op in het eerste register en stelt de condition code in op basis van het resultaat:

CC	Meaning
0	Result is zero
1	Result is negative
2	Result is positive
3	Result has overflowed (Het resultaat paste niet in het register)

Tabel 4.3: De mogelijkheden van de CC na de 'AR' instructie (z/Architecture Principles of Operation, 2022).

In het volgende voorbeeld wordt gekeken naar hoe de 'CR' instructie de CC aanpast.

Mnemonic	Operands	Opcode	Instruction
CR	R,R	19	COMPARE (32)

Tabel 4.4: De eigenschappen van de mnemonic 'CR' (2025).

Deze instructie vergelijkt de waarden in twee registers met elkaar zonder deze te veranderen en stelt de condition code in:

De CC heeft geen vierde mogelijkheid, de logische uitkomsten zijn gelijk, kleiner of groter.

¹Het Program Status Word (PSW) is een register met verschillende velden die informatie bijhouden over het programma dat wordt uitgevoerd.

²Een register is een kleine, snelle opslaglocatie binnen de processor die gebruikt wordt voor het tijdelijk bewaren en verwerken van gegevens en adressen.

CC	Meaning
0	Operand 1 = Operand 2
1	Operand 1 < Operand 2
2	Operand 1 > Operand 2

Tabel 4.5: De mogelijkheden van de CC na de 'CR' instructie (Ehrman, 2001).

4.3.1. Branching

De control-flow in HLASM wordt bepaald door instructies die het uitvoeringsadres van het programma wijzigen. De structuur van het programma verandert door sprongen naar andere posities in de code.

Deze sprongen worden aangegeven via branch-instructies. Branching omvat zowel conditionele als onvoorwaardelijke vertakkingen en sprongen naar subroutines.

Bij conditionele vertakkingen wordt gebruik gemaakt van de condition code (CC). Deze code wordt ingesteld door voorafgaande instructies, zoals vergelijkingen of rekenkundige operaties. De waarde van de CC bepaalt of de vertakkingsvoorwaarde voldaan is.

Wanneer niet aan de voorwaarde wordt voldaan, verloopt de uitvoering sequentieel en wordt de volgende instructie uitgevoerd. Wanneer er wel aan de voorwaarde wordt voldaan, wordt de normale uitvoering onderbroken en verdergezet op een andere locatie in het programma.

Dit gebeurt door het aanpassen van het adres van de volgende uit te voeren instructie in het Program Status Word (PSW). Het PSW bevat onder andere het adres van de huidige instructie. Door dit adres te wijzigen, wordt de uitvoering voortgezet vanaf een nieuw adres.

Naast conditionele vertakkingen bestaan er ook onvoorwaardelijke sprongen. Deze instructies voeren altijd een sprong uit, onafhankelijk van de condition code. Dit zorgt ervoor dat de sequentiële uitvoering expliciet wordt doorbroken.

Verder zijn er instructies die gebruikt worden voor het oproepen van subroutines. Eerst wordt het terugkeeradres opgeslaan, vervolgens wordt er vertakt naar een andere locatie in het programma. Op het einde kan de uitvoering hervat worden op de oorspronkelijke locatie.

Het doeladres van een sprong komt overeen met een label in de broncode. Tijdens de assemblage vertaalt de assembler deze labels naar concrete geheugenadressen.

```

1  ----+----1----+----2----+----3----+----4----+----5----+----6----+----7--
2          CR      R1,R2
3          BE      GELIJK
4          B       VERDER
5  *
6  GELIJK  AR      R1,R2
7  VERDER  SR      R1,R2

```

Listing 4.2: Voorbeeld van vertakkingen in HLASM.

In dit voorbeeld wordt eerst een vergelijking uitgevoerd tussen twee registers. Indien de condition code aangeeft dat beide waarden gelijk zijn, wordt er vertakt naar het label GELIJK. Als dit niet het geval is, wordt de volgende instructie uitgevoerd, namelijk een onvoorwaardelijke sprong naar VERDER.

4.3.2. Relative Branching J-instructies

Naast de klassieke branch-instructies bestaan er binnen HLASM ook relative branch-instructies. Deze instructies gebruiken een relatieve offset³ ten opzichte van de huidige positie in het programma om een sprong uit te voeren.

Een voorbeeld hiervan zijn de J-instructies, zoals J, JE en JNE. Deze instructies worden gebruikt voor respectievelijk onvoorwaardelijke en conditionele vertakkingen.

```

1  ----+----1----+----2----+----3----+----4----+----5----+----6----+----7--
2          CR      R1,R2
3          JE      GELIJK
4          J       VERDER
5  *
6  GELIJK  AR      R1,R2
7  VERDER  SR      R1,R2

```

Listing 4.3: Voorbeeld van relative branching met J-instructies.

De werking van deze instructies is gelijkaardig aan gewone branch-instructies. Ze wijzigen namelijk ook de control-flow van het programma door naar een andere positie in de code te springen.

³Een offset is een relatieve afstand ten opzichte van een bepaalde geheugenlocatie of instructiepositie.

4.3.3. Compare-and-Branch (CGIJ)-instructies

HLASM bevat ook compare-and-branch instructies. Deze combineren een vergelijking en een vertakking in één enkele instructie.

Een voorbeeld hiervan is de CGIJ-instructie. Deze instructie vergelijkt een registerwaarde met een onmiddellijke waarde en voert daarna eventueel een sprong uit afhankelijk van het resultaat.

```

1  -----1-----2-----3-----4-----5-----6-----7--
2          CGIJ  R1,10,NL,GROTER
3          AR    R2,R3
4  *
5  GROTER  SR    R2,R3

```

Listing 4.4: Voorbeeld van een CGIJ-instructie.

In dit voorbeeld wordt de waarde in register R1 vergeleken met de waarde 10. Indien de voorwaarde voldaan is, wordt er vertakt naar het label GROTER.

Deze instructies hebben rechtstreeks invloed op de control-flow van een programma omdat ze zowel een vergelijking uitvoeren als een mogelijke vertakking bepalen.

4.3.4. Instructies voor Control-Flow Mapping

Niet alle instructies hebben een invloed op de control-flow van een programma. Binnen HLASM kunnen deze instructies onderverdeeld worden in vier categorieën:

- conditionele vertakkingen,
- onvoorwaardelijke vertakkingen,
- subroutine-aanroepen,
- instructies die de condition code instellen.

Categorie	Mnemonic	Beschrijving
Conditionele vertakkingen	BE, BNE, BL, BH, BNH, BNL	Vertakken afhankelijk van de waarde van de condition code. Deze instructies geven meerdere mogelijke uitvoeringspaden.
Onvoorwaardelijke vertakkingen	B, BR	Vertakken altijd naar een opgegeven adres of label. De sequentiële uitvoering wordt hierbij onderbroken.
Subroutine vertakkingen	BAL, BAS, BASR	Slaan het terugkeeradres op en vertakken naar een subroutine. Deze instructies zijn gelijkaardig aan functieaanroepen.
Vergelijkingsinstructies	C, CR	Voeren een vergelijking uit en stellen de condition code in zonder de operanden te wijzigen.
Rekenkundige instructies	AR, SR	Voeren een berekening uit en passen de condition code aan op basis van het resultaat.
Relative branching	J, JE, JNE	Vertakken relatief ten opzichte van de huidige instructiepositie.
Compare-and-branch	CGIJ	Combineert een vergelijking en een vertakking in één instructie.

Tabel 4.6: Relevante HLASM-instructies voor control-flow mapping.

Voor de opbouw van een control-flowgrafiek is het voldoende om vertakkingsinstructies te detecteren. Deze bepalen expliciet de mogelijke paden doorheen het programma en vormen de verbindingen tussen knooppunten in de grafiek.

Instructies die de condition code instellen spelen een indirecte rol. Zij bepalen de uitkomst van conditionele vertakkingen, maar zijn niet noodzakelijk om de structuur van de control-flow te representeren.

Binnen deze bachelorproef ligt de focus op het detecteren van labels en klassieke branch-instructies, zoals B- en J-instructies. Labels worden gemodelleerd als knooppunten in de control-flowgrafiek, terwijl vertakkingen worden voorgesteld als verbindingen tussen deze knooppunten.

Meer complexe instructies, zoals compare-and-branch instructies, worden buiten de scope van de proof of concept gehouden.

5

Control-flow visualisatie

Dit hoofdstuk beschrijft hoe de control-flow van HLASM-programma's gevisualiseerd kan worden. Het doel van deze visualisatie is om beginnende ontwikkelaars een overzicht te geven van de mogelijke control-flow binnen een HLASM-programma. In HLASM is de programmastructuur niet onmiddellijk zichtbaar in de broncode. De control-flow moet daarom afgeleid worden op basis van labels, spronginstructies en de sequentiële uitvoering van instructies. Een grafische representatie maakt deze structuur beter interpreteerbaar.

De visualisatie wordt gerealiseerd in de vorm van een webapplicatie. Dit maakt de analyse en visualisatie laagdrempelig en interactief, zonder bijkomende installatievereisten. De gebruiker kan HLASM-code invoeren in de browser, waarna automatisch een control-flowdiagram wordt gegenereerd.

5.1. Requirements

Voor het ontwikkelen van een proof-of-concept worden zowel functionele als niet-functionele vereisten opgesteld. Deze vereisten bepalen welke onderdelen van HLASM-code worden geanalyseerd en hoe de visualisatie wordt opgebouwd.

5.1.1. Functionele vereisten

Voor de PoC zijn de volgende functionele vereisten gedefinieerd:

- Detectie van labels in de broncode, zodat sprongdoelen geïdentificeerd kunnen worden.
- Detectie van vertakkingsinstructies die de control-flow beïnvloeden, waaronder zowel conditionele als onvoorwaardelijke sprongen.
- Correcte mapping van labels naar knooppunten (nodes) in de control-flowgrafiek.
- Correcte mapping van sprongen naar verbindingen tussen knooppunten.

5.1.2. Niet-functionele vereisten

Naast de functionele vereisten zijn er ook niet-functionele vereisten opgesteld:

- Leesbaarheid: de gegenereerde diagrammen moeten een duidelijk overzicht geven van de control-flow, waarbij vertakkingen en uitvoeringspaden eenvoudig te volgen zijn.
- Correctheid: de gegenereerde Mermaid.js-code moet syntactisch correct zijn en zonder aanpassingen renderbaar zijn.
- Automatisatie: de visualisatie moet volledig automatisch gegenereerd worden op basis van de HLASM-broncode, zonder manuele tussenstappen.
- Toegankelijkheid: de toepassing wordt aangeboden als webapplicatie, zodat deze onmiddellijk bruikbaar is via een browser zonder installatie.
- Beperkingen: de implementatie richt zich op een afgebakende subset van HLASM-instructies.

5.2. Ontwerp

Het ontwerp van de visualisatietool bestaat uit een vertaling van HLASM-broncode naar een grafische representatie van de control-flow. Deze representatie wordt getoond als een flowchart.

De implementatie gebeurt als een webapplicatie. De logica wordt geïmplementeerd via JavaScript. Hierdoor blijft de toepassing eenvoudig en onafhankelijk van externe systemen.

De opbouw van de flowchart gebeurt in verschillende stappen:

- De ingevoerde broncode wordt opgesplitst in individuele instructies.
- Per instructie worden het label, de mnemonic en de operanden geïdentificeerd op basis van de kolomgebaseerde structuur.
- Alle labels worden verzameld en gekoppeld aan hun positie in de code.

Hierbij wordt onderscheid gemaakt tussen expliciete en impliciete control-flow:

- **Expliciete control-flow:** veroorzaakt door branch-instructies die naar een label springen.
- **Impliciete control-flow:** de standaard sequentiële uitvoering naar de volgende instructie.

5.2.1. Mapping van HLASM naar Mermaid.js

Om de control-flowgrafiek visueel weer te geven, wordt gebruik gemaakt van Mermaid.js. Hiervoor wordt een mapping gedefinieerd tussen HLASM-elementen en grafische elementen.

HLASM-element	Mnemonic	Mermaid.js representatie
Label	Niet van toepassing	Node in de flowchart
Sequentiële instructie	L, AR, SR, etc.	Verbinding naar volgende node
Onvoorwaardelijke sprong	B, BR	Edge naar doelnode
Conditionele sprong	BE, BNE, BH, BL	Vertakkende edge met conditie
Programma start	CSECT, START	Entry node
Programma einde	END	Eindnode

Tabel 5.1: Mapping van HLASM-elementen naar Mermaid.js

Bij conditionele sprongen ontstaan meerdere mogelijke paden. In de visualisatie worden deze weergegeven als vertakkingen met een label dat de voorwaarde aanduidt. Onvoorwaardelijke sprongen worden voorgesteld als directe verbindingen zonder extra voorwaarden.

Sequentiële instructies worden impliciet verbonden met de volgende instructie in de code, tenzij deze onderbroken worden door een spronginstructie.

Door deze mapping toe te passen kan de structuur van een HLASM-programma worden weergegeven als een flowchart, waarbij de nadruk ligt op de control-flow.

6

Proof-of-Concept

6.1. Architectuur van de applicatie

De proof-of-concept werd geïmplementeerd als een webapplicatie die HLASM-broncode omzet naar een control-flowdiagram in Mermaid.js-syntax. Omdat het een webapplicatie is, hoeft de gebruiker niets te installeren. De tool kan meteen gebruikt worden via de browser. Dit maakt het eenvoudig om de applicatie snel te testen en te gebruiken.

De werking van de applicatie bestaat uit drie stappen:

1. **Parsing:** de ingevoerde HLASM-broncode wordt opgesplitst in individuele instructies. Per regel worden label, mnemonic en operanden geïdentificeerd op basis van de kolomgebaseerde structuur.
2. **Control-flow analyse:** op basis van de gedetecteerde instructies wordt bepaald hoe de uitvoering van het programma kan verlopen.
3. **Visualisatie:** de structuur wordt omgezet naar Mermaid.js-syntax, die vervolgens wordt omgezet naar een flowchart.

6.1.1. Stap 1: Parsing

In deze eerste stap wordt de HLASM-code verwerkt tot een lijst van instructies. Lege lijnen en commentaar (lijnen die starten met een *) worden genegeerd. Voor elke lijn worden drie onderdelen bepaald:

- het label (indien aanwezig),
- de mnemonic (de instructienaam),
- het eerste operand (argument).

De parser maakt gebruik van de vaste kolomstructuur door specifieke delen van de lijn uit te lezen op vaste posities.

Na deze stap wordt elke lijn voorgesteld als een object met duidelijke velden. Dit maakt het eenvoudiger om de volgende stap uit te voeren.

```

1 function parseInstructions(hlasmCode) {
2   const instructions = [];
3
4   hlasmCode.split('\n').forEach((line, index) => {
5     if (!line || line.trim() === "" || line.trim().startsWith("*"))
6       return;
7
8     const padded   = line.padEnd(80, " ");
9     const label    = padded.substring(0, 8).trim();
10    const mnemonic = padded.substring(9, 14).trim().toUpperCase();
11
12    if (!mnemonic) return;
13
14    const operandField = padded.substring(15, 71).trim();
15    const operandMatch = operandField.match(/^(^[\s']+|^'[^']*')+$/);
16    const fullOperand  = operandMatch ? operandMatch[0] : "";
17    const firstOperand = fullOperand.split(',')[0].trim();
18
19    instructions.push({
20      lineNumber: index + 1,
21      raw: `${label.padEnd(8)} ${mnemonic.padEnd(5)} ${fullOperand}`
22    },
23      {
24        label,
25        mnemonic,
26        operand: firstOperand
27      }
28    );
29  });
30
31  return instructions;
32 }
```

Listing 6.1: De HLASM parser.

6.1.2. Step 2: Control-flow analyse

In deze stap wordt gekeken hoe het programma verloopt tijdens uitvoering.

De instructies worden gegroepeerd in blokken. Een blok is een reeks instructies die na elkaar uitgevoerd worden, zonder onderbreking.

Een nieuw blok start in de volgende situaties:

- wanneer een instructie een label bevat,
- wanneer er een sprong (branch) gebeurt,
- wanneer het programma eindigt (END).

Voor elk blok wordt ook bepaald hoe het blok eindigt. Dit kan bijvoorbeeld een sprong zijn naar een ander label, of een gewone overgang naar het volgende blok. Op deze manier ontstaat een structuur die beschrijft hoe de uitvoering van het programma kan verlopen.

```

1 function buildBlocks(instructions) {
2   let blockId = 0;
3   const blocks = [];
4   let currentBlock = createBlock(0);
5
6   instructions.forEach((instr, i) => {
7     if (instr.label && currentBlock.lines.length > 0) {
8       blocks.push(currentBlock);
9       currentBlock = createBlock(++blockId);
10    }
11
12    currentBlock.lines.push(instr);
13
14    if (instr.label) {
15      currentBlock.labels.push(instr.label);
16    }
17
18    const isBranch = isBranchInstruction(instr.mnemonic);
19    const isEnd = instr.mnemonic === "END";
20
21    if (isBranch || isEnd) {
22      currentBlock.exit = instr;
23      blocks.push(currentBlock);
24      currentBlock = createBlock(++blockId);
25    } else if (instructions[i + 1] && instructions[i + 1].label) {
26      currentBlock.exit = { mnemonic: "SEQUENTIAL", operand: "" };
27      blocks.push(currentBlock);
28      currentBlock = createBlock(++blockId);
29    }
30  });
31

```

```

32     if (currentBlock.lines.length > 0 && !blocks.includes(currentBlock)) {
33         blocks.push(currentBlock);
34     }
35
36     return blocks;
37 }

```

Listing 6.2: De code die instructies in blokken verdeelt.

6.1.3. Stap 3: Visualisatie

In de laatste stap wordt de gevonden structuur omgezet naar een visueel diagram. Eerst worden alle blokken voorgesteld als knopen (nodes). De inhoud van elk blok wordt weergegeven als tekst in de knoop.

Daarna worden verbindingen (pijlen) toegevoegd tussen de blokken:

- een gewone pijl voor een normale overgang,
- een pijl met label voor een conditionele sprong,
- een directe pijl voor een onvoorwaardelijke sprong.

Deze informatie wordt omgezet naar Mermaid.js-syntaxis. Mermaid.js zorgt er vervolgens voor dat deze tekst automatisch wordt weergegeven als een flowchart.

```

1 function blocksToMermaid(blocks) {
2     let mermaid = "%{init: {'themeVariables': {'fontFamily': 'monospace'}}}%\nflowchart TD\n";
3
4     blocks.forEach(block => {
5         const text = block.lines
6             .map(l => l.raw.replace(/\s+/, ' ').trim()).join("<br/>");
7         mermaid += `    ${block.id}["${text}"]\n`;
8     });
9
10    blocks.forEach((block, i) => {
11        const next = blocks[i + 1];
12        const exit = block.exit;
13
14        // we have reached the last block
15        if (!exit) return;
16
17        const m = exit.mnemonic;
18        const targetLabel = exit.operand;
19
20        if (isConditionalBranch(m)) {
21            const targetBlock = findBlockByLabel(blocks, targetLabel);
22            if (targetBlock) mermaid += `    ${block.id} -->|${m}| ${targetBlock.id}\n`;

```

```

23     if (next) mermaid += `    ${block.id} --> ${next.id}\n`;
24   } else if (isUnconditionalBranch(m)) {
25     const targetBlock = findBlockByLabel(blocks, targetLabel);
26     if (targetBlock) mermaid += `    ${block.id} --> ${targetBlock
27   .id}\n`;
28   } else if (m === "SEQUENTIAL" && next) {
29     mermaid += `    ${block.id} --> ${next.id}\n`;
30   }
31   });
32   return mermaid;
33 }

```

Listing 6.3: De conversie naar mermaid.js.

6.1.4. Gebruikersinterface

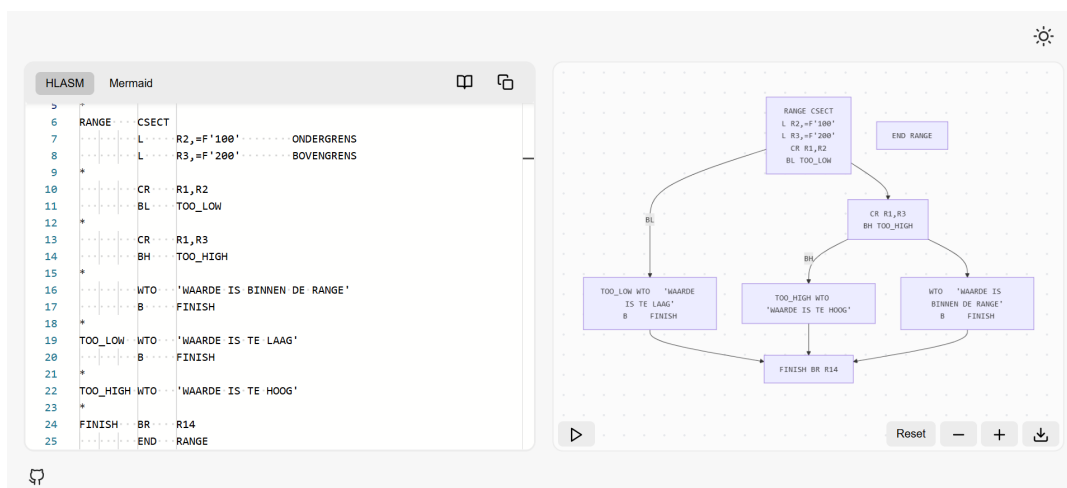
De gebruikersinterface van de applicatie is eenvoudig opgebouwd.

Er zijn twee onderdelen:

- een invoerveld waarin de gebruiker HLASM-code kan invoeren,
- een visualisatiepaneel waarin het diagram wordt weergegeven.

Wanneer de gebruiker code invoert, wordt het diagram automatisch bijgewerkt.

De gebruiker ziet dus onmiddellijk het resultaat van de analyse.



Figuur 6.1: Invoer van HLASM-code in de webapplicatie.

6.1.5. Gebruik van de applicatie

Het gebruik van de tool verloopt in enkele eenvoudige stappen:

1. De gebruiker plakt of typt HLASM-broncode in de editor.
2. De applicatie verwerkt de code automatisch.
3. Op basis van de analyse wordt een Mermaid.js-diagram gegenereerd.
4. Het diagram wordt onmiddellijk visueel weergegeven.

De gebruiker hoeft geen kennis te hebben van Mermaid.js. De visualisatie wordt volledig automatisch gegenereerd en helpt bij het begrijpen van de structuur van het programma.

6.2. Beperkingen van de implementatie

Omdat de implementatie gebruik maakt van een vereenvoudigde aanpak zijn er enkele beperkingen.

Ten eerste wordt geen volledige syntactische of semantische analyse uitgevoerd. De parser gaat ervan uit dat de invoer de standaard kolomstructuur volgt. Als dit niet het geval is, kan de interpretatie fout zijn.

Daarnaast worden macro-instructies niet uitgewerkt. Macros kunnen meerdere instructies bevatten, waardoor een deel van de control-flow niet zichtbaar is in het diagram.

Verder worden enkel directe sprongen naar labels ondersteund. Indirecte sprongen, bijvoorbeeld via registers (zoals `BR R14`), worden niet geanalyseerd.

Tot slot wordt de condition code niet inhoudelijk geïnterpreteerd.

Deze beperkingen zijn aanvaardbaar binnen deze proof-of-concept. De focus ligt op het zichtbaar maken van de globale structuur van het programma.

6.3. Evaluatie

De evaluatie onderzoekt of de tool helpt bij het begrijpen van HLASM-programma's. Alle deelnemers hadden geen voorkennis van HLASM. Ze kregen vooraf een korte uitleg over control-flow, met nadruk op labels en sprongen.

De evaluatie werd uitgevoerd met vier deelnemers. Elke deelnemer kreeg twee codefragmenten en moest:

- het startpunt identificeren,
- vertakkingen aanduiden,
- de globale structuur beschrijven.

Deze opdrachten werden uitgevoerd:

- zonder hulpmiddel,
- met gebruik van de tool.

6.3.1. Resultaten

Uit de evaluatie blijkt dat de tool vooral helpt om snel een overzicht te krijgen van de globale structuur van een programma.

De deelnemers konden met de visualisatie sneller zien hoe de verschillende delen van het programma met elkaar verbonden zijn en waar vertakkingen plaatsvinden. Dit maakte het eenvoudiger om de algemene flow te begrijpen.

Tegelijk werd duidelijk dat dit overzicht niet voldoende is om de code volledig te begrijpen. De deelnemers gaven aan dat het nog steeds nodig is om de volledige code lijn per lijn te bekijken om de werking correct te interpreteren.

De tool werd daarom niet gezien als een vervanging van manuele analyse, maar als een hulpmiddel dat het proces ondersteunt.

Daarnaast werd opgemerkt dat de meerwaarde van de tool vooral zichtbaar is bij grotere programma's. Bij kleinere codefragmenten is de structuur meestal eenvoudig genoeg om manueel te volgen, waardoor de visualisatie weinig extra toevoegt.

7

Conclusie

In deze bachelorproef werd onderzocht in welke mate statische analyse en Mermaid-diagrammen kunnen helpen bij het leren van HLASM door beginnende ontwikkelaars.

De centrale onderzoeksvraag luidde:

Hoe effectief versnellen statische analyse en Mermaid-diagrammen het leerproces van HLASM bij beginners?

Om deze vraag te beantwoorden werd eerst een literatuurstudie uitgevoerd naar mainframes, HLASM en de moeilijkheden die beginners ervaren bij assemblytalen. Daarna werd een analyse gemaakt van HLASM om te bepalen welke onderdelen belangrijk zijn voor de control-flow. Op basis hiervan werd een proof-of-concept ontwikkeld die HLASM-code omzet naar een diagram in Mermaid.js.

Uit de evaluatie blijkt dat de visualisatie vooral helpt om sneller een overzicht te krijgen van de structuur van een programma. De deelnemers konden makkelijker zien hoe verschillende delen van de code met elkaar verbonden zijn en waar vertakkingen plaatsvinden.

Tegelijk werd duidelijk dat deze visualisatie niet voldoende is om de code volledig te begrijpen. De deelnemers gaven aan dat het nog steeds nodig is om de code zelf te lezen en te analyseren. De tool versnelt dus niet het volledige leerproces, maar helpt vooral bij het begrijpen van de globale structuur.

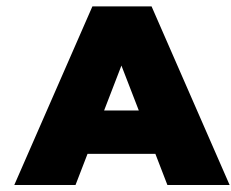
De tool wordt daarom best gezien als een aanvullend hulpmiddel, en niet als een vervanging van manuele analyse. Vooral bij grotere programma's biedt de visualisatie een duidelijke meerwaarde, omdat het moeilijker is om daar zelf snel een overzicht van te krijgen. Bij kleinere programma's is deze meerwaarde beperkter. Er moet rekening worden gehouden met de beperkingen van dit onderzoek. De proof-of-concept ondersteunt slechts een beperkt deel van HLASM en houdt geen

rekening met complexere constructies zoals macro's of indirecte sprongen. Daarnaast was de testgroep klein, waardoor de resultaten niet volledig veralgemeend kunnen worden.

Ondanks deze beperkingen toont dit onderzoek aan dat een eenvoudige visualisatietool nuttig kan zijn bij het analyseren van HLASM-code. Dit is relevant in de praktijk, waar het begrijpen van bestaande code vaak moeilijk is door een gebrek aan documentatie.

Voor toekomstig werk kan de tool verder uitgebreid worden met ondersteuning voor meer instructies en complexere situaties. Ook kan onderzocht worden hoe deze visualisatie geïntegreerd kan worden in bestaande ontwikkelomgevingen. Daarnaast kan een grotere evaluatie bijkomende inzichten geven.

Samengevat kan gesteld worden dat statische analyse in combinatie met een visuele voorstelling helpt om de structuur van HLASM-programma's sneller te begrijpen. De tool ondersteunt het leerproces, maar vervangt het niet.



Onderzoeksvoorstel

Het onderwerp van deze bachelorproef is gebaseerd op een onderzoeksvoorstel dat vooraf werd beoordeeld door de promotor. Dat voorstel is opgenomen in deze bijlage.

A.1. Inleiding

Tegenwoordig spelen mainframecomputers een cruciale rol in de dagelijkse activiteiten van bedrijven in de financiële sector, de gezondheidszorg, verzekeringen en tal van andere publieke en private ondernemingen. Deze bedrijven zijn afhankelijk van mainframecomputers en hebben dan ook de nood om deze zo optimaal mogelijk te onderhouden. Vermits veel ervaren mainframe-specialisten hun pensioen naderen, is er nood aan een nieuwe generatie mainframe-specialisten die het onderhoud van hun systemen kan waarborgen. Onder dit onderhoud vallen de bedrijfskritische functionaliteiten van mainframeapplicaties, die voornamelijk afhankelijk zijn van High-Level Assembler (HLASM). Deze legacycode is vaak tientallen jaren oud en minimaal gedocumenteerd. Voor de nieuwe generatie mainframeontwikkelaars vormt dit een uitdaging om deze legacycode te begrijpen en te onderhouden. Zij vertrouwen vaak enkel op manuele code reviews.

Door middel van een geautomatiseerde tool die statische analyse van HLASM-code visualiseert in een control-flowchart, wil dit onderzoek ondersteuning bieden aan beginnende mainframeontwikkelaars, specifiek binnen organisaties die afhankelijk zijn van legacy HLASM-code.

De probleemstelling van dit onderzoek luidt als volgt:

Hoe effectief versnellen statische analyse en Mermaid-diagrammen het leerproces van HLASM bij beginners?

Het doel van dit onderzoek is het ontwikkelen van een proof-of-concept (PoC) die

een representatieve subset van HLASM-instructies en labels statisch analyseert en deze omzet in een visueel control-flowdiagram in Mermaid.js-syntax.

Vooraleer de ontwikkeling van de PoC van start gaat, wordt een literatuurstudie uitgevoerd naar het probleemdomain en het oplossingsdomain.

Rond het probleemdomain worden volgende vragen bestudeerd:

- Wat is het belang van de mainframecomputer?
- Waarom is er nood aan het onderhouden van HLASM-code?
- Wat zijn de moeilijkheden bij het begrijpen van assemblytalen voor beginnende ontwikkelaars?

Rond het oplossings domein worden volgende vragen bestudeerd:

- Hoe helpt visualisatie bij het sneller begrijpen van programma's?
- Welke tools bestaan er al om HLASM-code te visualiseren?
- Welke HLASM parsers bestaan er?
- Welke HLASM-instructies zijn van noodzaak om de control-flow van de code in kaart te brengen?
- Wat is de beste manier om de kolom-gebaseerde structuur van HLASM te verwerken met een JavaScript-parser?
- Hoe vertaal je de vertakkingen in de HLASM-code naar correcte Mermaid.js-diagrammen?

A.2. Literatuurstudie

A.2.1. Belang van de mainframecomputer

Vaak worden mainframes beschreven als oude technologie en wordt gedacht dat ze niet meer relevant zijn. Ook wouden bedrijven graag mainframes vervangen door andere servers. Momenteel zijn mainframes nog steeds zeer actief en staan ze in voor kritische bedrijfsactiviteiten. Tegenwoordig worden mainframes ook geïntegreerd met moderne technologieën (Sagers e.a., 2018). Het onderzoek van Sagers e.a. (2018) wordt bevestigd door recent onderzoek waarin wordt aangetoond dat organisaties niet enkel gebruikmaken van legacyapplicaties, maar ook bezig zijn met het actief ontwikkelen van nieuwe applicaties (Shaik, 2024).

Mainframes zijn dus een zeer relevante en moderne technologie, die dagelijks draait. Omdat ervaren mainframeprofessionals hun pensioen naderen is er een behoefte aan een nieuwe generatie mainframespecialisten die het onderhoud van deze systemen kunnen waarborgen (Waites & Ketterer, 2013).

A.2.2. Nood aan onderhoud van HLASM-code

Bedrijfskritische functionaliteiten van mainframeapplicaties zijn vaak afhankelijk van High Level Assembler (HLASM), waardoor onderhoud van deze legacycode van groot belang is. Als de aangeroepen HLASM-code faalt, dan faalt het volledige programma (Zaytsev, 2020). Uit een onderzoek naar softwareonderhoud blijkt dat het begrijpen van bestaande code een groot deel van de moeite neemt tijdens onderhoud. Vooral bij beginnende ontwikkelaars gaat het merendeels van hun tijd aan het begrijpen van de functionaliteit, dit is omdat zij minder ervaring hebben (Xia e.a., 2018). Dit is vooral het geval bij legacycode zoals HLASM, omdat daar de documentatie vaak beperkt is.

A.2.3. Moeilijkheden voor beginnende ontwikkelaars

Assemblytalen zijn zeer gedetailleerd, als ontwikkelaar definieer je elke kleine stap van het programma, wat ingewikkeld kan zijn voor beginners. Uit onderzoek blijkt dat naarmate de ervaring toeneemt, het gemakkelijker wordt om de functionaliteit van code te begrijpen (Feigenspan e.a., 2012).

Binnen het onderwijs hebben studenten vooral moeite met de componenten die de logische volgorde van het programma bepalen. Dit zijn de componenten die de control flow van het programma definiëren. Bij het begrijpen van de control flow maken beginners vaak fouten en ervaren ze vaak verwarring (Kawash, 2024). Hieruit kan worden geconcludeerd dat de control flow van code een moeilijkheid is onder beginnende ontwikkelaars.

A.2.4. Effectiviteit van visualisatietechnieken

Visualisatietechnieken worden vaak gebruikt om code beter te begrijpen en worden ook erkend als technieken die sneller en meer inzicht geven in codefunctionaliteiten. Visualisaties worden vaak gebruikt om de structuur van code te representeren (Storey, 2005). Bij assemblytalen kan visualisatie vooral helpen, aangezien de code vaak is opgebouwd uit zeer gedetailleerde instructies en kan worden gegroepeerd in componenten (Baldwin e.a., 2009).

A.2.5. Bestaande visualisatietools en parsers

Een bestaande tool die kan worden gebruikt voor de visualisatie van HLASM-code is FermaT. Deze tool is vooral gericht op het begrijpen en migreren van legacycode. Het is een zeer krachtige en zware tool die veel functionaliteiten heeft (Ward, 2001). Ook biedt IBM een HLASM parser aan, die onderdeel maakt van IBM Developer for z/OS. Deze parser is bedoeld voor diepe analyse en gebruik met andere tools (IBM, g.d.).

Het doel van dit onderzoek is het ontwikkelen van een lichte tool die HLASM-code omzet naar een high-level control-flowdiagram visualisatie in Mermaid.js-syntax. Deze tool is specifiek bedoeld voor educatieve doeleinden, om beginnende ont-

wikkelaars te ondersteunen. Ook ligt de focus enkel op een beperkte subset van control-flow-instructies, in tegenstelling tot bestaande tools.

A.2.6. Control Flow in High Level Assembler

High Level Assembler (HLASM) wordt gezien als een Second Generation Language (2GL), de instructies in HLASM komen rechtstreeks overeen met processor-commando's. Er zijn dus geen abstracties zoals functies of objecten. Deze abstracties zijn wel terug te vinden in talen met een hoger generatieniveau zoals COBOL. HLASM biedt daardoor veel controle over het fysieke systeem, maar dat maakt de code vaak moeilijker te lezen en te onderhouden (Kornelis, 2003).

Om de control-flow van HLASM te definiëren kan er gefocust worden op bepaalde elementen die sprongen of vertakkingen maken:

- Een directe sprong: Instructies zoals B (Branch) sturen het programma direct naar een ander codeblok (*High Level Assembler for z/OS & z/VM & z/VSE: HLASM V1R6 Programmer's Guide*, [g.d.](#)).
- Een conditionele sprong: Door middel van vergelijkingen (C voor Compare) of condities (BE voor Branch on Equal) verandert de route van het programma (*High Level Assembler for z/OS & z/VM & z/VSE: HLASM V1R6 Programmer's Guide*, [g.d.](#)).
- Een subroutine: Bij het aanroepen van een subroutine worden instructies zoals BAL of BAS gebruikt. Het terugkeren vanuit een subroutine gebeurt via een BR-instructie (Ward, 2013).

Deze specifieke instructies kunnen een idee geven van de control-flow van het programma.

A.2.7. JavaScript Parsing

Binnen HLASM wordt er gewerkt met een vaste kolom-gebaseerde indeling. Elk onderdeel van de code staat op een vaste plek. Elke kolom heeft een eigen functie: kolommen 1-8 zijn voor labels, kolommen 10-14 voor de instructie en vanaf kolom 16 staan de operanden. Na kolom 71 wordt alles beschouwd als commentaar of wordt het genegeerd (*HLASM V1R6 Language Reference*, [g.d.](#)).

JavaScript is zeer nuttig om tekst te parsen, het is namelijk erg goed in het doorzoeken en herkennen van patronen (Flanagan, 2011). Voor de PoC is een regel-voor-regel aanpak voldoende. Hierbij zal de parser elke regel doorlopen op basis van de vaste kolomposities. Zo kunnen de labels en instructies herkend worden die bijdragen aan de control-flow. De verzamelde elementen worden vervolgens gemapt naar Mermaid.js-elementen.

A.2.8. Mermaid.js en flowcharts

Met Mermaid.js worden flowcharts en andere structuren vanuit tekst automatisch gevisualiseerd (Sveidqvist & Jain, 2021). Dit maakt het makkelijk om een mapping te maken van HLASM-elementen naar een visuele representatie. Het gebruik van flowcharts om complexe software begrijpelijk te maken is een bekend middel in de informatica (Ensmenger, 2016).

Een flowchart maakt gebruik van blokken en pijlen om een route te definiëren. In de context van informatica worden flowcharts vaak gebruikt om de route van een programma weer te geven (Charntaweechun & Wangsiripitak, 2006). In het geval van HLASM kunnen bijvoorbeeld spronginstructies worden weergegeven als pijlen die het programma brengen naar een ander codeblok.

A.3. Methodologie

Deze bachelorproef is een toegepast en technisch onderzoek waarin een oplossing wordt gezocht voor het onderhouden en begrijpen van legacy HLASM-broncode. Het onderzoek start met een literatuurstudie naar het probleemdomein en het oplossingsdomein. Ook wordt de control flow van HLASM-broncode bestudeerd, waarna een subset van representatieve HLASM-instructies en labels geselecteerd worden. Na de literatuurstudie start de ontwikkeling van een technische proof-of-concept (PoC) die de geselecteerde subset visueel omzet in een control-flowchart in Mermaid.js-syntax.

A.3.1. Fase 1: Oriëntatie en Literatuurstudie

Tijdens deze fase wordt gefocust op het aanscherpen van de probleemstelling en het oplossingsdomein. Dit gebeurt door middel van een literatuurstudie waarin dieper wordt ingegaan op de deelvragen:

- Wat is het belang van de mainframecomputer?
- Waarom is er nood aan het onderhouden van HLASM-code?
- Wat zijn de moeilijkheden bij het begrijpen van assemblytalen voor beginnende ontwikkelaars?
- Wat is de effectiviteit van visualisatietechnieken voor het sneller begrijpen van programma's?
- Wat zijn de bestaande visualisatietools binnen High Level Assembler?
- Welke HLASM-instructies zijn van noodzaak om de control-flow van de code in kaart te brengen?
- Wat is de beste manier om de kolom-gebaseerde structuur van HLASM te verwerken met een JavaScript-parser?

- Hoe vertaal je de vertakkingen in de HLASM-code naar correcte Mermaid.js-diagrammen?

Deze fase loopt van 20 december 2025 tot 1 maart 2026 en resulteert in een uitgewerkte literatuurstudie en methodologie. Het resultaat bevat concreet een theoretisch onderzoek naar de probleemstelling en oplossingsdomein, en een afgebakende scope van HLASM-instructies en labels voor de verdere ontwikkeling van de technische oplossing.

A.3.2. Fase 2: Requirementsanalyse

Tijdens de literatuurstudie (tussen januari 2026 en maart 2026) wordt een requirementsanalyse uitgevoerd. Het doel van deze fase is om concrete en meetbare eisen voor de proof of concept en de evaluatie vast te leggen.

Binnen deze fase beginnen we eerst met het specifiek definiëren van de doelgroep. Uit de doelgroep wordt gezocht naar 6 à 10 deelnemers die instaan voor de evaluatie van de PoC. Voor de evaluatie worden ook HLASM-codefragmenten verzameld die typische control-flowconstructies hebben zoals labels, conditionele takken en subroutines-aanroepen.

Ook worden de succescriteria van de PoC vastgesteld:

- correcte detectie van de geselecteerde HLASM-instructies en labels;
- correcte omzetting naar Mermaid.js-flowcharts;
- meetbare tijdswinst bij het begrijpen van de control flow in vergelijking met een manuele analyse.

A.3.3. Fase 3: Implementatie en Evaluatie

Tijdens de implementatie- en evaluatiefase wordt de proof-of-concept uitgewerkt en iteratief geëvalueerd. Deze fase vindt plaats tussen 2 maart 2026 en 4 mei 2026. De ontwikkeling gebeurt volgens een scrum-werkwijze om de effectiviteit van de PoC te waarborgen. Met behulp van de subset die in de vorige fase werd gedefinieerd, wordt een JavaScript-gebaseerde parser ontwikkeld die HLASM-code statisch analyseert. De parser identificeert instructies en labels aan de hand van patroonherkenning in de structuur van HLASM-broncode en zet deze via een mapping om naar Mermaid.js-syntax.

Concreet worden tijdens deze fase de volgende aspecten ontwikkeld:

- de logische opbouw van de parser op basis van patroonherkenning in de HLASM-broncodestructuur;
- een mapping van de geselecteerde HLASM-instructies en labels naar Mermaid.js-elementen;

- een evaluatie van de PoC op correctheid en tijdswinst.

Tijdens de ontwikkeling wordt de PoC iteratief geëvalueerd via een vergelijkende studie met handmatige code reviews. De evaluatie wordt uitgevoerd door 6 à 10 vrijwillige deelnemers, de helft van de deelnemers gebruikt de PoC om codefragmenten uit te leggen en de andere helft doet dit zonder de PoC. Beide groepen maken gebruik van dezelfde codefragmenten en worden getimed op het moment waarop zij zelf aangeven dat ze de code begrijpen.

De evaluatie loopt tijdens de implementatie, van 1 april 2026 tot 4 mei 2026.

A.3.4. Fase 4: Resultaat en Verdediging

De laatste fase loopt van 4 mei 2026 tot juni 2026. In deze fase worden de onderzoeksresultaten en evaluatieresultaten verwerkt tot een conclusie.

Specifiek worden de volgende aspecten besproken:

- de sterktes en beperkingen van de PoC;
- mogelijke uitbreidingen voor toekomstig onderzoek.

Deze fase resulteert in de finale bachelorproef die wordt ingediend op 29 mei 2026, de verdediging en de proof-of-concept met de bijbehorende documentatie.

A.3.5. Tijdsplanning

Elke fase heeft een tijdsperiode waarin de specifieke resultaten moeten worden opgeleverd voor de volgende fase. Zoals te zien in figuur ??, zijn ook de administratieve deliverables opgenomen.

A.4. Verwacht resultaat, conclusie

Het verwachte resultaat van deze studie is een proof of concept (PoC), die in staat is een subset van representatieve HLASM- en labels om te zetten in visuele stappen geïnterpreteerd in Mermaid.js-syntax. De PoC brengt door middel van statische HLASM-broncodeanalyse de control flow in kaart. Met deze PoC wordt verwacht beginnende mainframeontwikkelaars te ondersteunen met het begrijpen van een onbekend stuk HLASM-broncode. De evaluatie van het verwachte resultaat wordt uitgevoerd door middel van een vergelijking met de traditionele, handmatige manieren van code reviews.

Bibliografie

- (2018). <https://chevrotain.io/docs/>
- (2025, oktober). IBM Corporation. <https://www.ibm.com/docs/en/hla-and-tf/1.6.0?topic=instructions-table-all-supported>
- avishek-sen-gupta. (2025). Tape/Z.
- Baldwin, J., Myers, D., Storey, M.-A., & Coady, Y. (2009). Assembly Visualization and Analysis: An Old Dog CAN Learn New Tricks! <https://ecs.wgtn.ac.nz/foswiki/pub/Events/PLATEAU/2009Program/plateau09-baldwin.pdf>
- Bosworth, E. L. (2009). Chapter 2 Structure of an IBM Mainframe Assembler Language Program. http://edwardbosworth.com/My3121Textbook_HTM/MyText3121_Ch02_V02.htm
- Cárdenas, G. O., & Aponte, J. (2017). Evaluating the Graph-based Visualization Technique: A Controlled Experiment. *Enfoque UTE*, 8(1), 201–216. <https://www.redalyc.org/journal/5722/572262176015/html/>
- Charntaweehun, K., & Wangsiripitak, S. (2006). Visual Programming using Flowchart. *2006 International Symposium on Communications and Information Technologies*, 1062–1065. <https://doi.org/10.1109/ISCIT.2006.339940>
- Contributors, M. (2023, september). JavaScript. <https://developer.mozilla.org/en-US/docs/Web/javascript>
- Debbabi, M., Salcic, Z., Biglari-Abhari, M., & Malik, A. (g.d.). Control-Flow Graph - an overview | ScienceDirect Topics. <https://www.sciencedirect.com/topics/computer-science/control-flow-graph>
- Ehrman, J. (2001). *High Level Assembler: Toolkit Feature Technical Overview Synopsis*. <https://public.dhe.ibm.com/software/websphere/awdtools/hlasm/s8166a.pdf>
- Ehrman, J. (2004). *Extending the Life Cycle of Legacy Applications*. <https://public.dhe.ibm.com/software/websphere/awdtools/hlasm/S8132JEA.pdf>
- Ensmenger, N. (2016). The Multiple Meanings of a Flowchart. *51*(3), 321–351. Verkregen januari 25, 2026, van <http://www.jstor.org/stable/44667617>
- Feigenspan, J., Kästner, C., Liebig, J., Apel, S., & Hanenberg, S. (2012). Measuring programming experience. *2012 20th IEEE International Conference on Program Comprehension (ICPC)*, 73–82. <https://doi.org/10.1109/ICPC.2012.6240511>
- Flanagan, D. (2011). *JavaScript: The Definitive Guide* (6th). O'Reilly Media.

- High Level Assembler for z/OS & z/VM & z/VSE Language Reference Version 1 Release 6* [SC26-4940-07]. (g.d.). https://www.ibm.com/docs/en/SSLTBW_2.2.0/pdf/asmr1022.pdf
- High Level Assembler for z/OS & z/VM & z/VSE: HLASM VIR6 Programmer's Guide* [SC26-4941-07]. (g.d.). IBM Corporation.
- HLASM VIR6 Language Reference* [SC26-4940-06]. (g.d.). IBM Corporation.
- IBM. (g.d.). *HLASM Parser* [IBM Developer for z/OS documentation]. Verkregen januari 27, 2026, van <https://www.ibm.com/docs/en/developer-for-zos/16.0.x?topic=files-hlasm-parser>
- Kawash, J. (2024). Where Do Students Struggle Most in a First Course on Assembly Language? *Proceedings of the 26th Western Canadian Conference on Computing Education*. <https://doi.org/10.1145/3660650.3660652>
- Kornelis, A. F. (2003). HLASM - Why assembler? <https://bixoft.nl/english/why.htm>
- Sagers, G., Ball, K., Hosack, B., Twitchell, D., & Wallace, D. (2018). The Mainframe Is Dead. Long Live the Mainframe! *AIS Transactions on Enterprise Systems*, 2(1). <https://www.aes-journal.com/index.php/ais-tes/article/view/6>
- Shaik, S. (2024). Importance of Mainframe in Modern Era of Technologies. *Journal of Engineering and Applied Sciences Technology*, 1–3. [https://doi.org/https://doi.org/10.47363/jeast/2024\(6\)257](https://doi.org/https://doi.org/10.47363/jeast/2024(6)257)
- Sneed, H., & Verhoef, C. (2019). Re-implementing a legacy system. *Journal of Systems and Software*, 155. <https://doi.org/https://doi.org/10.1016/j.jss.2019.05.012>
- Storey, M.-A. (2005). Theories, methods and tools in program comprehension. *IEEE Software*, 22(3), 36–45.
- Sun, W., Fang, C., Miao, Y., You, Y., Yuan, M., Chen, Y., Zhang, Q., Guo, A., Chen, X., Liu, Y., & Chen, Z. (2023, december). Abstract Syntax Tree for Programming Language Understanding and Representation: How Far Are We? <https://doi.org/https://doi.org/10.48550/arXiv.2312.00413>
- Sveidqvist, K., & Jain, A. (2021). *The official guide to Mermaid.js: Create complex diagrams and beautiful flowcharts easily using text and code*. Packt Publishing.
- Waites, S., & Ketterer, J. (2013). Lack of Mainframe Programmers a Critical Issue for IT Organizations A Review of the Literature. *International Journal of Business, Humanities and Technology*, 3(7). https://ijbht.thebrpi.org/journals/Vol_3_No_7_September_2013/3.pdf
- Ward, M. (2013). Assembler restructuring in FermaT. *2013 IEEE 13th International Working Conference on Source Code Analysis and Manipulation (SCAM)*, 147–156. <https://doi.org/10.1109/SCAM.2013.6648196>
- Ward, M. (2020). FermaT. <https://www.gkc.org.uk/fermat.html>

- Ward, M. (2001). The FermaT assembler re-engineering workbench. *Proceedings IEEE International Conference on Software Maintenance. ICSM 2001*, 659–662. <https://doi.org/10.1109/ICSM.2001.972783>
- Xia, X., Bao, L., Lo, D., Xing, Z., Hassan, A. E., & Li, S. (2018). Measuring Program Comprehension: A Large-Scale Field Study with Professionals. *IEEE Transactions on Software Engineering*, 44(10), 951–976. <https://doi.org/10.1109/TSE.2017.2734091>
- Yalamanchili, C. M. (2022). Technical Insights into IBM High-Level Assembler (HLASM) in z/OS Systems. *International Journal of Multidisciplinary Research and Growth Evaluation*, 3(1), 1155–1162. <https://doi.org/https://doi.org/10.54660/ijmrge.2022.3.1.1155-1162>
- z/Architecture Principles of Operation [SA22-7832-13.pdf]. (2022, mei). https://www.ibm.com/docs/en/module_1678991624569/pdf/SA22-7832-13.pdf
- Zaytsev, V. (2020). Journal of Object Technology Published by AITO -Association Internationale pour les Technologies Objets Vadim Zaytsev. Modelling of Language Syntax and Semantics: The Case of the Assembler Compiler. *Journal of Object Technology*, 19(1), 1–22. <https://grammarware.net/text/2020/hlasm.pdf>